AI201 PA 2 Report
Naive Bayes Spam Filter

Adriane Jone A. Abunda (2025-22450)
Date of Submission: March 8, 2026

1 INTRODUCTION

Unsolicited bulk email, commonly known as spam, remains a persistent challenge that consumes network resources, storage capacity, and introduces significant cybersecurity risks through phishing vectors. Early filtering systems relied heavily on knowledge engineering, utilizing static, hand-crafted rules to flag malicious messages based on known keyword lists. However, as spammers continuously adapted their techniques to bypass these, the field transitioned toward probabilistic machine learning models.

Unlike rule-based systems, machine learning algorithms offer dynamic filtering by capturing the statistical distribution of tokens within both spam and legitimate (ham) messages. Among these algorithms, the Naive Bayes classifier remains highly utilized due to its computational efficiency, linear scaling, and baseline performance, despite its strong assumption of conditional feature independence [4].

Spam filtering is fundamentally distinct from standard text categorization due to the asymmetry in misclassification costs. While misclassifying a spam message as legitimate (a False Negative) is a minor annoyance, misclassifying a critical legitimate email as spam (a False Positive) can result in professional, financial, or personal consequences. Therefore, spam filters must be inherently biased toward maintaining near-perfect precision while maximizing recall.

This report explores the construction of a custom Naive Bayes classifier evaluated on the TREC06 Public Spam corpus. Furthermore, it investigates the impact of Lambda Smoothing to mitigate the zero-frequency problem. Finally, it explores the dimensionality reduction techniques proposed by Johan Hovold [1], specifically evaluating the effect of Mutual Information (MI) ranking on attribute selection. By identifying the most highly discriminative words and discarding noise, this report demonstrates how to construct a lightweight, computationally efficient classifier suitable for real-time, end-user applications.

2 OBJECTIVES

The primary objective of this programming assignment is to design, implement, and evaluate a Naive Bayes spam classifier entirely from scratch, without reliance on pre-existing machine learning libraries such as `scikit-learn`.

Specific objectives include:

1. **Data Pipeline Construction:** To develop a data parser that reads, cleans, and partitions the TREC06 Public Spam corpus into disjoint training (70%) and testing (30%) sets.

2. **Baseline Implementation:** To construct a base Naive Bayes classifier using raw count statistics and prior probabilities to establish a baseline performance metric.

3. **Lambda Smoothing:** To implement a modified class-conditional likelihood formula utilizing Lambda Smoothing (λ). The objective is to empirically evaluate five distinct values of λ (2.0, 1.0, 0.5, 0.1, 0.005) to identify the best Lambda value that maximizes both precision and recall.

4. **Dimensionality Reduction via Mutual Information:** To implement the attribute selection methodology suggested by Hovold (2005). This involves filtering out high-frequency non-discriminative words, calculating the Mutual Information for the remaining vocabulary, and evaluating the classifier using only the top 200 most informative words.

5. **Performance Analysis:** To compare the baseline, smoothed, and reduced-vocabulary models using standard information retrieval metrics, specifically Precision, Recall, and the F1 Score, and to interpret the trade-offs of these configurations.

3 METHODOLOGY

3.1 Data Preparation and Parsing

The dataset utilized for this assignment is a subset of the TREC06 Public Spam Corpus, a globally recognized benchmark for spam filtering algorithms. The dataset consists of thousands of individual email files organized into subdirectories, with a master `labels` file mapping each file path to its respective class ('spam' or 'ham').

The data was initially loaded and shuffled using a fixed random seed to ensure reproducibility, followed by a strict 70-30 train-test split. Document parsing was performed using `Latin-1` character encoding to handle anomalous byte characters common in raw email data. To normalize the text and extract meaningful features, a word was strictly defined as any sequence of alphabetic characters `[a-zA-Z]` delimited by a leading whitespace and a trailing whitespace, comma, or period. All extracted tokens were converted to lowercase to ensure case-insensitivity, resulting in a set of unique words per document.

Class Imbalance

The training corpus shows a notable class imbalance, comprising 17,353 spam documents (65.54%) and 9,122 ham documents (34.46%), resulting in an approximate 1.9:1 spam-to-ham ratio. In many discriminative classification models, such as Support Vector Machines or k-Nearest Neighbors, this imbalance can introduce bias toward the majority class, often necessitating corrective techniques such as oversampling (e.g., SMOTE) or undersampling of the majority class [4].

However, the Naive Bayes classifier is a model that inherently accounts for class imbalance through its prior probability term $P(\omega)$. The prior is estimated directly from the training distribution:

$$P(\omega = \text{spam}) = \frac{N_{\text{spam}}}{N_{\text{spam}} + N_{\text{ham}}} = \frac{17353}{26475} \approx 0.6554 \tag{1}$$

This prior is multiplied into the posterior during classification, meaning the model is already calibrated to the true class distribution of the corpus. Rebalancing the dataset prior to training would distort this prior away from the true distribution, potentially degrading classification performance. Therefore, no resampling was applied, and the class distribution was preserved throughout all experiments. The validity of this decision is supported by the experimental results of the baseline model having a recall of only 0.9218, confirming discriminative behavior.

3.2 The Naive Bayes Mathematical Framework

Spam detection is a two-category classification problem. The Naive Bayes classifier operates on Bayes' Theorem, which calculates the posterior probability of a class given a set of features (words). The algorithm assigns a document d to the class $\omega \in \{\text{spam, ham}\}$ that yields the Maximum A Posteriori (MAP) estimate.

Given a document represented by a feature vector $x = (x_1, x_2, ..., x_d)$, the posterior probability is:

$$P(\omega|x_1, x_2, ..., x_d) = \frac{P(\omega) \prod_{i=1}^{d} P(x_i|\omega)}{\sum_{\omega} P(\omega) \prod_{i=1}^{d} P(x_i|\omega)} \tag{2}$$

In practice, multiplying thousands of small probabilities results in arithmetic underflow, a hardware limitation of floating-point representation. To circumvent this, the programmatic implementation calculates the log-likelihood. By applying the natural logarithm, the product of probabilities transforms into a sum of log-probabilities, resulting in the following decision rule:

$$\hat{\omega} = \arg\max_{\omega} \left(\log P(\omega) + \sum_{i=1}^{d} \log P(x_i|\omega) \right) \tag{3}$$

3.3 Lambda Smoothing

The baseline model calculates the conditional probability $P(x_i|\omega)$ strictly based on the exact counts of word x_i in the documents of class ω within the training set. However, if a word in the test set never appeared in the spam training set, its conditional probability becomes zero. Since log(0) resolves to negative infinity, a single unseen word forces the model to misclassify the document.

To resolve this, Lambda Smoothing (a generalized form of Laplace Smoothing) was implemented. It adds a small smoothing factor λ to the numerator and scales the denominator by $\lambda|V|$, where $|V|$ is the size of the entire vocabulary:

$$P(x_i|\omega) = \frac{\left[\sum_{D \in D_\omega} I(x_i \in D)\right] + \lambda}{|D_\omega| + \lambda|V|} \tag{4}$$

This ensures unseen words are assigned a negligible but non-zero probability, allowing the algorithm to evaluate the document based on all available tokens rather than failing on a single unknown word.

3.4 Attribute Selection via Mutual Information (Hovold)

The standard Naive Bayes classifier treats all words equally, yet words such as "the" and "is" offer zero discriminative value. Following the methodology outlined by Hovold [1], an attribute selection mechanism was implemented using Mutual Information (MI). MI quantifies how much information the presence (or absence) of a word contributes to making the correct classification:

$$MI(X;C) = \sum_{x \in \{0,1\}} \sum_{c \in C} P(x,c) \log \left(\frac{P(x,c)}{P(x)P(c)} \right) \tag{5}$$

Applying MI directly to the entire vocabulary yields suboptimal results because very frequent tokens dominate the upper rankings. A pre-filtering step was implemented to forcefully remove the 200 most frequent words in the corpus before calculating MI. Once filtered, MI was calculated for all remaining words. The vocabulary was then truncated to the top 200 words ranked by their global MI score. Each of the 200 selected words was subsequently categorized as either spam-indicative or ham-indicative by comparing its document frequency rate across both classes, a

word is labeled spam-indicative if it appears in a higher proportion of spam documents than ham documents, and ham-indicative otherwise. In this experiment, the resulting split happened to be exactly 100 spam-indicative and 100 ham-indicative words, though this outcome is a property of the corpus distribution rather than a constraint imposed by the selection algorithm.

3.5 Evaluation Metrics

To measure the effectiveness of the classifier configurations, True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) were tracked. The primary evaluation metrics used are Precision, Recall, and the F1 Score:

$$Precision = \frac{TP}{TP + FP} \qquad Recall = \frac{TP}{TP + FN} \tag{6}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{7}$$

4 EXPERIMENTAL RESULTS

4.1 Corpus Statistics

The 70-30 split of the TREC06 subset yielded a training set of 26,475 documents and a testing set of 11,347 documents. The training set was skewed toward spam, comprising 17,353 spam messages ($P(\text{spam}) \approx 0.6554$) and 9,122 ham messages ($P(\text{ham}) \approx 0.3446$). Parsing the training documents resulted in an un-filtered vocabulary size of $|V| = 82,476$ unique words.

4.2 Baseline Evaluation (No Smoothing)

The baseline classifier was tested on the 11,347 test documents without smoothing applied. As shown in the confusion matrix in Figure 1, the results demonstrated near-perfect precision but compromised recall due to a high volume of false negatives.

- **Precision:** 0.9991
- **Recall:** 0.9218
- **F1 Score:** 0.9589

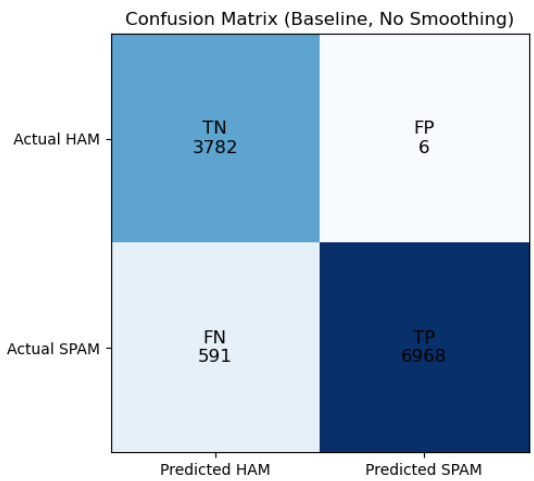

Figure 1: Confusion Matrix for the Baseline classifier (No Smoothing).

4.3 Lambda Smoothing Optimization

To optimize the classifier, five different values of λ were tested on the full vocabulary of 82,476 words. The results are summarized in Table 1 and visualized in Figure 2.

Table 1: Classifier Performance across various λ values

Lambda (λ)	**Precision**	**Recall**	**F1 Score**
2.0	0.9910	0.9862	0.9886
1.0	0.9947	0.9840	**0.9893**
0.5	0.9962	0.9743	0.9852
0.1	0.9990	0.9393	0.9682
0.005	0.9996	0.9045	0.9496

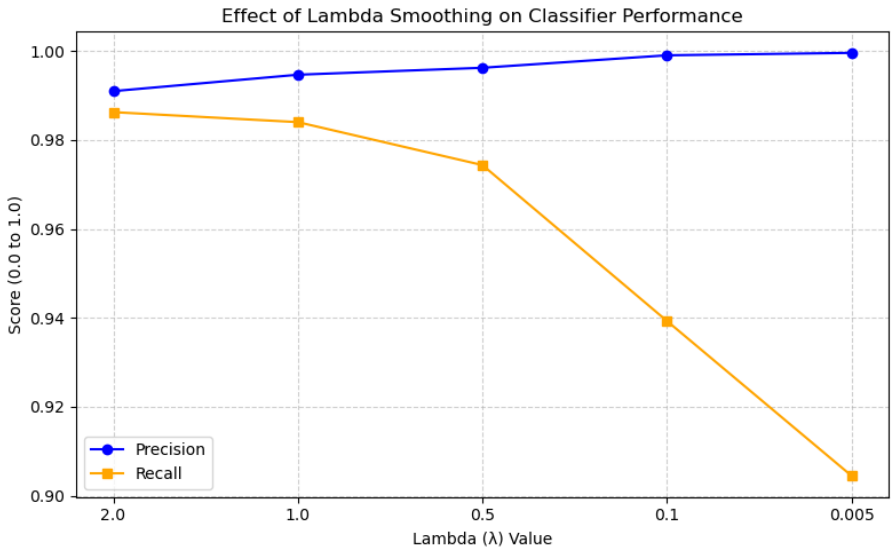

Figure 2: Effect of various Lambda (λ) smoothing values on Precision and Recall.

The optimal hyperparameter was determined to be $\lambda = 1.0$ (standard Laplace Smoothing). The confusion matrix for this configuration is provided in Figure 3.

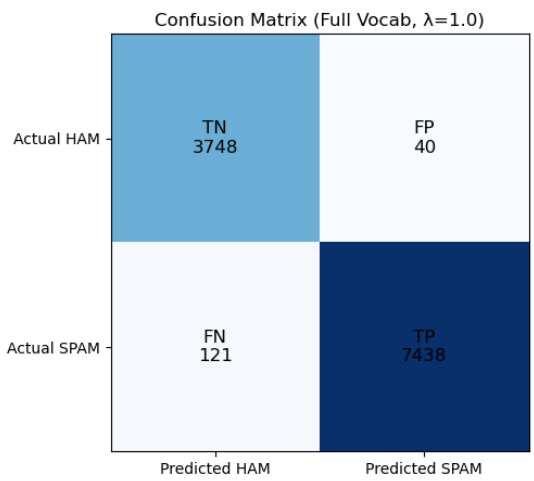

Figure 3: Confusion Matrix for the optimal smoothed classifier (Full Vocabulary, $\lambda = 1.0$).

4.4 Reduced Vocabulary (Hovold MI Ranking)

Applying the Hovold MI ranking required the removal of the 200 most frequent words. Subsequent MI calculation successfully identified the most informative attributes.

- Top 5 Spam-Indicative Words: *z, devmail, ra, foxmail, stock.*
- Top 5 Ham-Indicative Words: *university, handyboard, dmdx, ascii, ribbon.*

When the classifier was restricted to only use these top 200 words, utilizing the previously identified best $\lambda = 1.0$, the results were as follows (see Figure 4 for the confusion matrix distribution):

- **Precision:** 0.9731
- **Recall:** 0.9376
- **F1 Score:** 0.9550

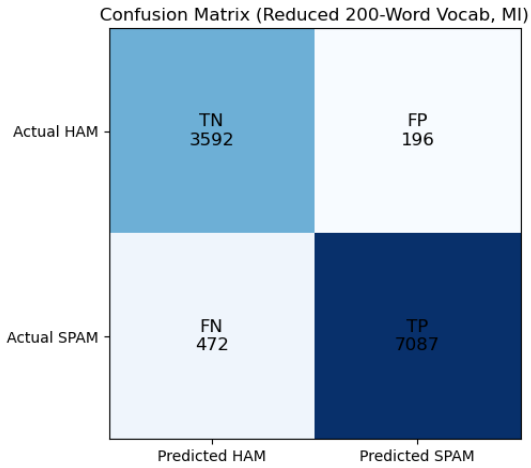

Figure 4: Confusion Matrix utilizing the Reduced 200-Word Vocabulary via MI Ranking.

5 ANALYSIS AND DISCUSSION OF RESULTS

5.1 The Zero-Frequency Problem in the Baseline Model

The baseline Naive Bayes model produced strong precision (0.9991) but a noticeably lower recall (0.9218). This disparity is a direct consequence of the zero-frequency problem inherent to un-smoothed probabilistic models. In the baseline implementation, if a test document contains a token absent from the spam training set, its conditional probability $P(w_i|\text{spam})$ evaluates to zero.

Because the algorithm sums log-probabilities, a value of zero resolves to negative infinity, mathematically overriding all other probabilistic evidence in the message. Consequently, a single out-of-vocabulary token causes the model to automatically classify the document as 'ham,' regardless of how many highly indicative spam tokens are present. This strict penalty accounts for the large volume of False Negatives visible in Figure 1.

5.2 The Lambda Smoothing Optimization

The introduction of additive smoothing successfully resolved the baseline limitation by allocating a small probability mass to unseen words. As observed in Table 1 and Figure 2, the choice of λ dictates the severity of this redistribution.

When under-smoothing is applied ($\lambda = 0.005$), the model regresses toward the baseline, resulting in a sharp drop in recall (0.9045) because the penalty for unseen words remains too harsh. Conversely, using a higher value like $\lambda = 2.0$ still produces excellent results (F1 = 0.9886) but slightly over-smooths the data. It adds a larger synthetic constant to the distributions, diluting the actual token frequency learned from the training set and causing a slight drop in precision compared to $\lambda = 1.0$. The empirical data confirms that Laplace Smoothing ($\lambda = 1.0$) provides the optimal balance for the TREC06 corpus.

5.3 Qualitative Assessment of Token Semantics

Following Hovold's methodology [1], removing the 200 most frequent words prior to calculating Mutual Information proved to be an essential step. Without this pre-filtering, MI rankings are typically dominated by recurring HTML formatting tags (e.g., "href", "td"), basic English stop words, and single characters that span across both classes.

By removing those noise, the MI metric successfully isolated the highly context-specific vocabulary of the TREC06 dataset. The highest-ranking legitimate words (e.g., university, ascii, files, linux, professor) suggest the 'ham' dataset originates primarily from a technical or academic environment. The top spam indicators (e.g., devmail, promotional, discreet, hoodia) align well with commercial email pushing software and pharmaceutical sales.

It is worth noting, however, that the pre-filtering step did not fully eliminate all non-semantic tokens. The word z remained after the frequency filter was applied and ranked as the top spam-indicative word (MI = 0.003595). This is a residual artifact, single-character tokens like z carries a skewed class distribution due to HTML formatting patterns or encoded content in message headers. While z does not carry semantic meaning as a spam signal, its class-conditional imbalance produces a high MI score. This limitation suggests that future implementations should supplement frequency-based filtering with a minimum word-length constraint (e.g., discarding tokens shorter than two or three characters) to eliminate such artifacts entirely. Despite this isolated residual, the overall MI ranking produced a vocabulary that is coherent, confirming that the attribute selection methodology was effective.

5.4 The Dimensionality Reduction Trade-off

The final comparison between the Full Vocabulary model and the Reduced Vocabulary model, summarized in Figure 5, illustrates a core engineering trade-off: balancing computational efficiency against predictive accuracy.

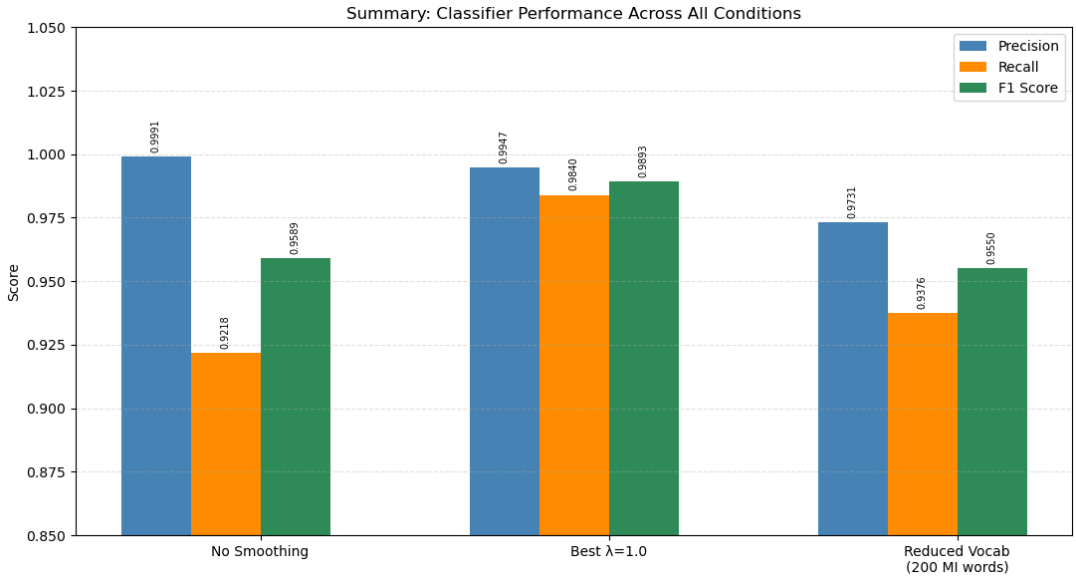

Figure 5: Final comparison of classifier performance across the Baseline, Optimal Smoothed, and Reduced Vocabulary configurations.

The Full Vocabulary model achieved an F1 Score of 0.9893 utilizing 82,476 unique features. By applying Hovold's Mutual Information ranking, the feature space was compressed to just 200 words, a dimensionality reduction of 99.76%. Despite this severe truncation, the model experienced only a marginal 3.4% decrease in its F1 score.

In production-level spam filters where processing latency and memory allocation are restricted, evaluating 82,000 probabilities per email is computationally expensive and memory-intensive. By strictly relying on the most discriminative words, an efficient classifier was created that still catches the vast majority of spam while maintaining excellent precision (0.9731). This validates the use of attribute selection in text classification tasks, proving that massive feature spaces are not strictly necessary if the remaining features contain high Mutual Information.

5.5 Error Analysis: False Positives and False Negatives

Although the optimal smoothed model ($\lambda = 1.0$) achieved excellent metrics, it is important to analyze where the classifier failed. Of the 11,347 test documents, the model generated 40 False Positives and 121 False Negatives.

False Positives typically occur when a legitimate email contains a high density of words that share overlapping distributions with spam. A legitimate newsletter regarding a software update or a banking alert may heavily utilize tokens like "free," "update," "click," or "account." Because Naive Bayes assumes feature independence, it fails to capture the syntactic and grammatical context of these words, and multiplies their high individual spam probabilities until they overpower the legitimate baseline probability.

Conversely, False Negatives often arise due to adversarial evasion techniques employed by spammers. These include intentional misspellings to evade dictionary checks (e.g., "l0an" instead of "loan", or "cRypt0" instead of "crypto"), inserting invisible HTML text (word salad), or utilizing image-based spam where the payload is hidden within an attached graphic rather than the plaintext body. Because the unigram Naive Bayes model relies entirely on exact string matching within the text body, it is inherently blind to these structural anomalies.

6 CONCLUSION

This assignment successfully demonstrated the mathematical formulation, Python programming implementation, and empirical evaluation of a custom Naive Bayes classifier from scratch. The baseline model highlighted the vulnerability of raw probabilistic classifiers to the zero-frequency problem, resulting in a degraded recall. Through Laplace smoothing ($\lambda = 1.0$), the classifier was effectively stabilized to handle out-of-vocabulary terms, resulting in an F1 score of 0.9893 in the TREC06 Public Spam corpus.

Furthermore, the application of Mutual Information (MI) for attribute selection proved to be a highly effective methodology for dimensionality reduction. By filtering out frequent noise and retaining only the top 200 most informative words, the algorithm achieved a 99.76% reduction in feature space. This resulted in an efficient model that suffered only a marginal 3.4% drop in performance, emphasizing the value of feature quality over feature quantity in production environments.

While the implemented classifier performs admirably, future iterations could explore several improvements:

- **Minimum Word-Length Filtering.** The frequency-based pre-filtering step used prior to MI ranking did not fully eliminate all non-semantic tokens. The word z ranked as the top spam-indicative word (MI = 0.003595) despite carrying no meaningful semantic content, a residual artifact caused by its disproportionate appearance in spam emails due to HTML formatting patterns in raw message headers. Supplementing the frequency filter with a minimum word-length constraint, such as discarding any token shorter than two or three characters, would eliminate such artifacts and further improve the quality of the selected attribute set.

- **N-gram Feature Expansion.** Future implementations could explore the inclusion of bigrams and trigrams (e.g., capturing "buy now" or "click here" as single features). While this increases memory requirements, n-grams possess significantly higher information content and are harder for spammers to successfully bypass [1].

- **Cost-Sensitive Classification.** Implementing a cost-sensitive weighting parameter would allow the model to force a much higher confidence threshold before labeling a message as spam, further safeguarding legitimate correspondence against false positives, a critical consideration given the asymmetric cost of misclassifying legitimate email in real-world deployments.

7 REFERENCES

[1] Johan Hovold, Naive Bayes Spam Filtering using Word Position Attributes, In *Conference on Email and Anti-Spam*, Stanford University, 2005.

[2] Tom M. Mitchell, Machine Learning, McGraw-Hill, 1997.

[3] Ion Androutsopoulos, John Koutsias, Konstantinos V. Chandrinos, and Constantine D. Spyropoulos, An experimental comparison of naive bayesian and keyword-based anti-spam filtering with personal e-mail messages, In *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 160-167, 2000.

[4] Mehran Sahami, Susan Dumais, David Heckerman, and Eric Horvitz, A Bayesian Approach to Filtering Junk E-mail. In *Learning for Text Categorization: Papers from the 1998 Workshop*, pp. 55-62, Madison, Wisconsin. AAAI Technical Report WS-98-05., 1998.